

Pythonの総整理

さて、ここまでAI班の資料を読み進めてくれた皆さん、本当にお疲れ様です！

このセクションは、これまでに学んだ重要な基本要素を一度ここで整理し、いつでも見返せるようにまとめておきましょう。見て覚えるのはプログラミングではありません。実際に実践しながら、忘れていたときに随時調べながら身体で覚えていきましょう。

基本のおさらい

1. 変数：データに名前をつける箱

- **役割**：数値や文字列などのデータに名前（変数名）をつけ、一時的に保存したり、後で再利用したりするためのものです（AI班2.md）。
- **作り方**： `変数名 = 値`（例： `my_score = 100` , `user_name = "AIチャレンジャー"` ）。
- **命名のコツ**：
 - 中身がわかるような、意味のある名前をつける（例： `age` 、 `total_price` ）。
 - Pythonでは、単語間をアンダースコア `_` でつなぐ「スネークケース」（例： `favorite_food` ）が一般的です。
 - 数字で始めない、予約語（ `if` , `for` など）は使えない、大文字・小文字は区別される、といったルールも思い出しましょう。

2. データ型：情報の種類を見極める

プログラムで扱うデータには様々な種類があります。Pythonは賢いので自動で判断してくれますが、主なものを理解しておくことは非常に重要です。

- **int（整数）**： `10` , `0` , `-25` のような小数点のない数値（AI班2.md）。
- **float（浮動小数点数）**： `3.14` , `9.8` , `-0.01` のような小数点のある数値（AI班2.md）。
- **str（文字列）**： `"こんにちは"` や `'Pythonが好き！'` のように、文字の並びをシングルクォートまたはダブルクォートで囲んだもの（AI班2.md, AI班3.md）。
 - **インデックス**：文字列内の各文字には、先頭から0, 1, 2... という番号（住所）がついています。 `my_string[0]` のようにして1文字を取り出せます。マイナスインデックス（ `[-1]` で末尾）も便利でしたね。
 - **スライス**： `my_string[開始:終了:ステップ]` の形で、文字列の一部を切り出せます。 `[::-1]` で逆順にするテクニックも思い出しましょう。
 - **便利なメソッド**：
 - `len(文字列)`：文字数を数える。

- `.upper()` , `.lower()` : 大文字・小文字変換。
 - `.replace(古い文字列, 新しい文字列)` : 文字列の置換。
 - `.find(探す文字列)` : 文字列を探して最初のインデックスを返す（見つからなければ-1）。
 - `.split(区切り文字)` : 文字列を区切り文字で分割し、リストにする。
 - `区切り文字列.join(文字列のリスト)` : リストの要素を連結して一つの文字列にする。
- **f-string (フォーマット済み文字列リテラル)** : `f"名前: {name}, 年齢: {age}"` のように、文字列内に変数の値を簡単に埋め込みます。 `{変数=}` の形でデバッグに役立つ表示もできましたね！(AI班3.md)
- **bool (ブール値)** : `True` (真) と `False` (偽) の2つの値だけを持つ型。プログラムの条件判断の基本です (AI班5.md)。
- **コレクション (複数のデータをまとめて扱う型)** (AI班4.md):
 - **list (リスト)** :
 - 特徴: `[10, "apple", True]` のように、複数のデータを **順番に** 格納でき、中身の **変更も可能** (ミュータブル)。角括弧 `[]` で作ります。
 - 主な操作: インデックスやスライスでのアクセス、`append()` (末尾に追加), `insert()` (指定位置に挿入), `pop()` (取り出して削除), `remove()` (値を指定して削除), `sort()` (並び替え) など。
 - **dict (辞書)** :
 - 特徴: `{"name": "高専太郎", "age": 18}` のように、**キー** と **値** のペアでデータを管理します。順番よりもキーで情報にアクセスするのが得意で、中身の **変更も可能**。波括弧 `{}` で作ります。
 - 主な操作: `辞書[キー]` での値の取得・設定、`keys()` (全キー取得), `values()` (全値取得), `items()` (全キーと値のペア取得), `get(キー, デフォルト値)` (安全な値の取得) など。
 - **tuple (タプル)** :
 - 特徴: `(10, 20, 30)` のように、複数のデータを **順番に** 格納できますが、一度作ると中身の **変更は不可能** (イミュータブル)。丸括弧 `()` で作ります。
 - 関数の戻り値で複数の値が返されるときや、辞書のキーとして (変更不可なため) 使われることがあります (AI班6.mdで少し触れました)。
- **型の確認** : `type(変数名)` で、その変数にどんな型のデータが入っているかを確認できましたね (AI班2.md)。

3. 演算子 : データを加工する

データを使って計算したり、比較したりするための記号です。

- **算術演算子** : `+` (足し算), `-` (引き算), `*` (掛け算), `/` (割り算、結果はfloat), `//` (切り捨て除算), `%` (余り), `**` (べき乗) (AI班2.md)。
- **比較演算子** : `==` (等しい), `!=` (等しくない), `<` (より小さい), `>` (より大きい), `<=` (以下), `>=` (以上)。結果は `True` か `False` (AI班5.md)。

- **論理演算子** : `and` (かつ), `or` (または), `not` (ではない)。複数の条件を組み合わせる時に使います (AI班 5 .md)。
- **文字列の演算** : `+` (連結), `*` (繰り返し) (AI班 2 .md, AI班 3 .md)。
- **`in` 演算子** : `要素 in コレクション` の形で、リストやタプルに特定の要素が含まれているか、文字列に部分文字列が含まれているか、辞書に特定のキーが存在するかを `True` / `False` で確認できます (AI班 4 .md, AI班 5 .md)。

4. `input()` 関数 : ユーザーとの対話

- `user_input = input("何か入力してください: ")` のようにして、プログラム実行中にユーザーからキーボード入力を受け取ることができます (AI班 2 .md)。
- **重要** : `input()` で受け取った値は **必ず文字列 (`str`) 型** になります。数値として扱いたい場合は、`int(user_input)` や `float(user_input)` のように型変換が必要です。

5. 制御構文 : プログラムの流れを操る

プログラムの処理の順番をコントロールするための命令です (AI班 5 .md)。

- **条件分岐 (`if` , `elif` , `else`)** :
 - `if` 条件式A:
 - 条件式AがTrueの時の処理
 - `elif` 条件式B:
 - 条件式AがFalseで、条件式BがTrueの時の処理
 - `else`:
 - 上のどの条件もFalseだった時の処理
 - 条件式には、比較演算子や論理演算子、`bool` 型の値を直接使います。
- **繰り返し (ループ)** :
 - **`for` ループ** :
 - `for` 変数 `in` イテラブルオブジェクト:
 - 繰り返したい処理
 - リスト、文字列、タプル、`range(数)` (連続した数値の生成)、辞書の `.items()` (キーと値のペア) など、順番に取り出せるもの (イテラブル) と一緒に使います。
 - `enumerate(イテラブル)` でインデックスと値を同時に、`zip(イテラブル1, イテラブル2)` で複数のイテラブルを同時に処理できましたね。
 - **`while` ループ** :
 - `while` 条件式:
 - 条件式がTrueの間、繰り返したい処理

- ループ内で条件式がいつか `False` になるように変数を操作しないと、**無限ループ** に陥るので注意！（停止は `Ctrl+C`）

- **ループ制御**：

- `break`：ループを途中で強制的に抜ける。
- `continue`：現在の回の残りの処理をスキップして、次の回の処理に進む。
- ループの `else` 節：ループが `break` で中断されずに最後まで正常に完了した場合に実行される処理。

6. 関数 (`def`)：処理をまとめる魔法の呪文

同じような処理を何度も書くのは大変ですよね。そんな時、処理をひとまとめにして名前をつけ、再利用可能にするのが「関数」です (AI班6.md)。

- **役割**：

- **部品化**：プログラムを小さな機能単位に分割できる。
- **再利用性**：同じ処理を何度も呼び出せる。
- **可読性**：コードがスッキリし、何をしているか分かりやすくなる。

- **定義**：`def 関数名(引数1, 引数2, ...):`

- 関数の処理内容
- `return` 戻り値

- **呼び出し**：`関数名(値1, 値2, ...)`

- **引数 (ひきすう)**：関数に渡す情報。

- **位置引数**：順番通りに渡される。
- **キーワード引数**：`引数名=値` の形で、順番を問わず渡せる。
- **デフォルト引数値**：`def my_func(name, country="日本"):` のように、引数に初期値を設定できる。

- **戻り値 (もどりち)**：関数が処理結果を返すためのもの。`return` を使う。

- 複数の値を `return name, age` のようにカンマで区切って返すと、それらは **タプル** としてまとめて返されます。
- `return` がない、または `return` だけの関数は `None` を返します。

- **スコープ**：変数が使える範囲のこと。関数内で定義された変数は基本的にその関数内だけで使える「ローカル変数」。関数の外で定義されたものは「グローバル変数」（安易な変更は注意）。

- **docstring (ドキュメンテーション文字列)**：`def` 文の直後に `"""関数が何をやるかの説明"""` のように書くことで、関数の説明書を残せます。`help(関数名)` で見れます。

7. 標準ライブラリ：Pythonの便利な工具箱

Pythonには、最初からたくさんの便利な機能（モジュール）が「標準ライブラリ」として用意されています（AI班6.md）。

- **使い方** : `import ライブラリ名` や `from ライブラリ名 import 特定の機能` のようにして読み込んでから使います。
- **例** :
 - `math` : 平方根 (`math.sqrt()`)、円周率 (`math.pi`) など、数学的な計算。
 - `random` : 乱数の生成 (`random.randint(a, b)` でaからbの整数、`random.choice(リスト)` でリストからランダム選択など)。
 - `datetime` : 日付や時刻の取得・操作 (`datetime.datetime.now()` で現在日時、`.strftime()` で書式指定など)。
- これらはほんの一部です。Pythonの公式ドキュメントなどでどんなライブラリがあるか見てみるのも面白いですよ！「こんなことできないかな？」と思ったら、まずは標準ライブラリに便利なものがないか探してみるのが良い習慣です。

8. コレクション型の使い分け再確認

- **リスト (list)** : 順番が重要で、後から中身を変えたり、要素の数を変えたりしたい場合に。（例：やることリスト、テストの点数一覧）
- **辞書 (dict)** : データに意味のある名前（キー）をつけて管理したい、関連する情報をまとめて扱いたい場合に。順番は基本気にしない（Python 3.7+では挿入順が保持されますが、それに依存しない方が良いでしょう）。（例：個人のプロフィール、設定情報）
- **タプル (tuple)** : 順番が重要だけど、一度決めた中身を変更されたくない（変更できないようにしたい）場合に。関数の戻り値で複数の値を安全に返したい時などにも使われます。

これからの学習に向けて：ライブラリと新しい「型」

ここまで学んできた基本は、いわばPythonという言語の「文法」や「基本的な単語」です。これから皆さんがNumPyのようなより専門的なライブラリを学んでいくと、それぞれのライブラリが独自に定義した新しい「型」や「オブジェクト」にたくさん出会うことになります。

例えば、`datetime` ライブラリを使ったとき、`datetime.datetime.now()` が返すものは、単なる数値や文字列ではなく、`datetime` オブジェクトという特別なものでしたね。これには年、月、日、時、分、秒といった情報がまとまって入っていて、`.year` や `.strftime()` のような専用の操作（メソッド）ができました。

NumPyを学ぶと、`ndarray`（N次元配列）という、大量の数値を効率的に扱うための非常に強力な「型」が登場します。これも、リストとはまた違った特徴や便利な操作方法を持っています。

ここが大事な心構えです！ これらの新しい「型」や「オブジェクト」が出てきても、慌てる必要はありません。

1. **全てを暗記しようとしないうこと** : 大切なのは、「このライブラリ（またはオブジェクト）は何をするためのものか」「どんな情報を持っていて、どんな操作ができるのか」という **基本的な考え方** を掴むことです。

2. **ドキュメントや資料を活用すること**：細かい使い方やメソッドの名前は、その都度、この資料や公式ドキュメント、インターネットで調べて確認すれば大丈夫です。
3. **これまでの知識が土台になること**：実は、これまでに学んだリストや辞書の操作方法（インデックスやスライスでのアクセス、キーを使ったアクセス、`len()` で長さを調べるなど）の考え方が、新しいオブジェクトを理解する上での大きな助けになることが多いです。「あ、この操作はリストのあの操作に似てるな」とか「このメソッドは辞書のキーで値を取ってくるのに似てるな」といった具合に、類推が効く場面がたくさんあります。

基本的なデータ型や制御構文、関数の知識というしっかりとした土台があれば、新しいライブラリや「型」も、それらを応用する形でスムーズに理解していくことができるはずです。

まとめ：基本を固めて、新しい世界へ

プログラミング学習の際には、何度もエラーにぶち当たります。でも、そんな時もあせらず、ただ粘り強く調べながら、試行錯誤しながら続けられる人は強くなっていきます。今後は、より発展的なライブラリ（Numpy, matplotlib, Torchなどなど）を学んでいきますが、この心構えで頑張ってください！